尚硅谷大数据项目之电商实时数仓（电商数据仓库系统）

（作者：尚硅谷研究院）

版本：V5.0

第 1 章 部署前准备工作

在/etc/profile.d/my_env.sh 中配置

```
export HADOOP_CLASSPATH=`hadoop classpath`
```

修改 yarn-site.xml 配置　container 最小和最大值以及 nm 分配的内存

```
<property>
   <name>yarn.scheduler.minimum-allocation-mb</name>
   <value>1024</value>
</property>
<property>
   <name>yarn.scheduler.maximum-allocation-mb</name>
   <value>2048</value>
</property>
<property>
   <name>yarn.nodemanager.resource.memory-mb</name>
   <value>4096</value>
</property>
```

修改 hadoop 下的 capacity-scheduler.xml 配置

```
这里意思是集群中可用于运行应用程序主机的最大资源百分比 - 控制并发活动应用程序的数量。每个队列的限制与其队列容量和用户限制成正比。官方文档默认是每个队列最大使用 10%的资源，把这里的 0.1 按需更改最大使用的资源数就行了
<property>
   <name>yarn.scheduler.capacity.maximum-am-resource-percent</name>
   <value>1</value>
</property>
```

修改 flink 客户端配置

```
jobmanager.memory.process.size: 1024m
taskmanager.memory.process.size: 1024m
taskmanager.numberOfTaskSlots: 2
```

第 2 章 手动部署

1. **将 gmall-realtime 父模块 install 到本地仓库**
2. **将 realtime-common 模块 install 到本地仓库**
3. **打包 dwd 下单和 dws 省份下单模块以及 Common 包上传到 flink 目录下**
4. **启动 zk、kafka、maxwell、hdfs、hbase、doris、redis、yarn 环境**
5. **启动历史服务 mapred --daemon start historyserver**

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

6. 提交 DwdTradeOrderDetail、DwsTradeProvinceOrderWindow 应用

-独立分窗口启动

```
bin/flink run-application \
-t yarn-application \
-d \
-Djobmanager.memory.process.size=1024mb \
-Dtaskmanager.memory.process.size=1024mb \
-Dtaskmanager.numberOfTaskSlots=2 \
-c com.atguigu.gmall.realtime.app.dwd.db.DwdTradeOrderDetail \
/opt/module/flink-1.17.0/gmall2023-realtime-1.0-SNAPSHOT.jar
```

```
bin/flink run-application \
-t yarn-application \
-d \
-Djobmanager.memory.process.size=1024mb \
-Dtaskmanager.memory.process.size=1024mb \
-Dtaskmanager.numberOfTaskSlots=2 \
-c com.atguigu.gmall.realtime.app.dws.DwsTradeProvinceOrderWindow \
/opt/module/flink-1.17.0/gmall2023-realtime-1.0-SNAPSHOT.jar
```

10.打包 publisher 并上传运行

11.修改内网穿透 natapp 映射的地址

改为 hadoop102:8070

12.sugar 修改空间映射

13.运行模拟生成日志的 jar 包，查看效果

```
Doris 内存大小限制修改
show VARIABLES like '%mem_limit%'

SET GLOBAL exec_mem_limit = 4096934592;
Yarn 查看错误日志
yarn logs -applicationId xxxx
```

第 3 章 项目部署及上线

3.1 StreamPark 部署

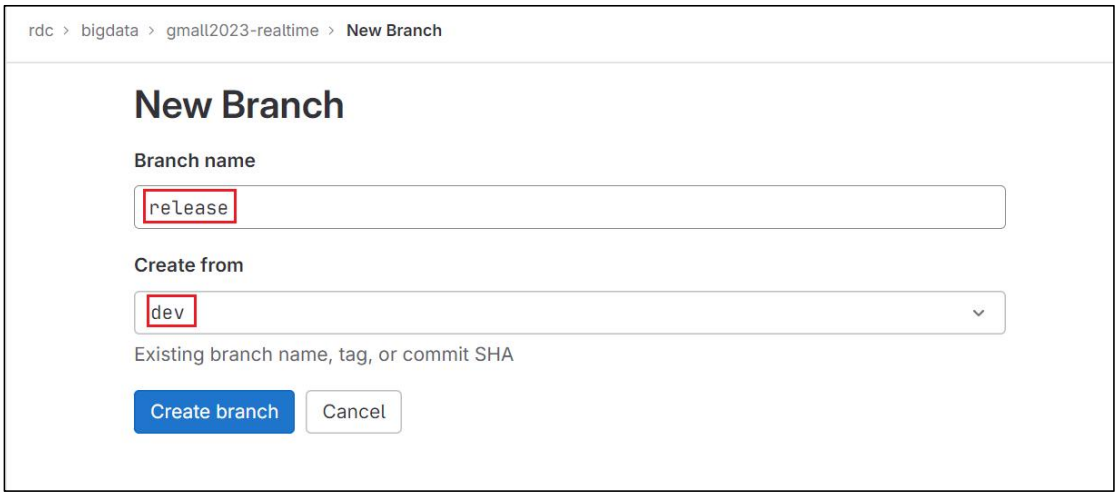
尚硅谷大数据项目之实时数仓_项目规

3.2 业务代码部署测试

本项目按照 GitFlow 工作流开发，为简化流程，开发全部在 dev 分支完成，没有新建 feature 分支。此时，所有业务代码已开发完成，在发布上线之前，我们要创建 release 分支，在该分支进行测试，测试通过则正式发布。

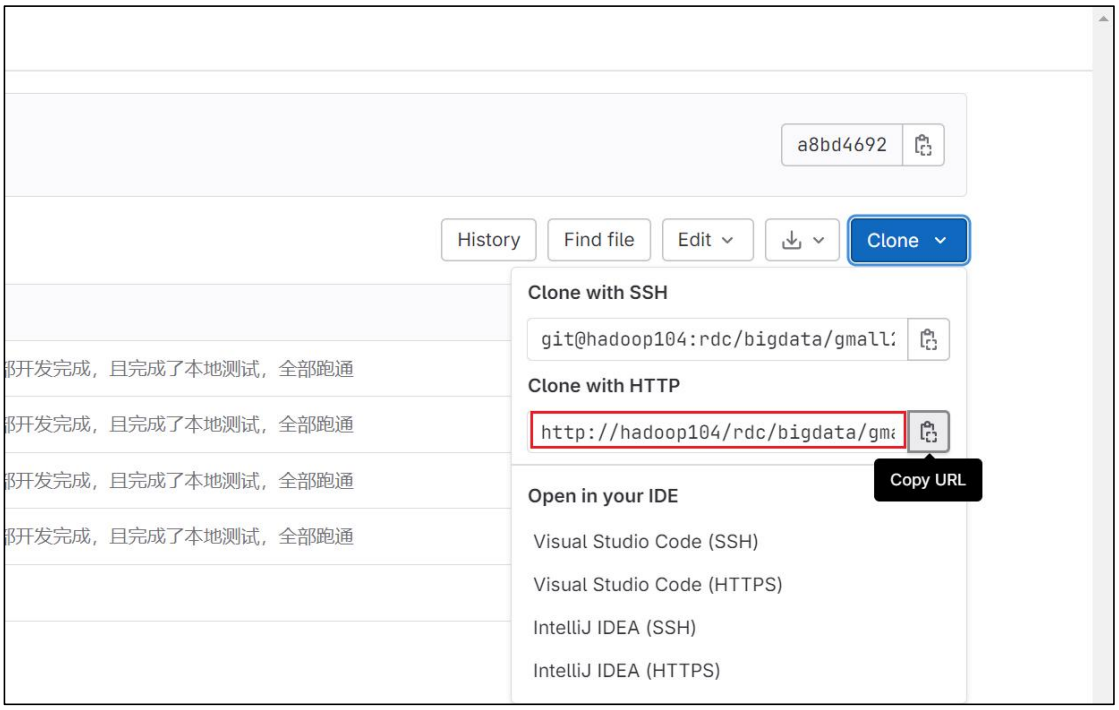
3.2.1 新建 release 分支

假设预发布分支的创建和后续的测试工作均由开发者 atguigu 完成，登录该账户，创建 release 分支，如下。

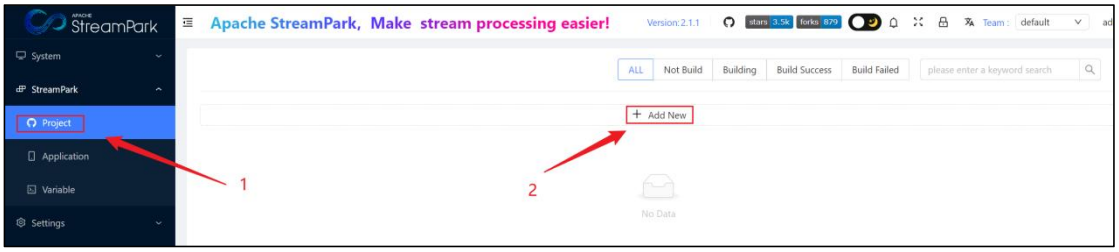


3.2.2 在 StreamPark 中新建项目

1）查看 GitLab 仓库地址



2）新建项目并配置

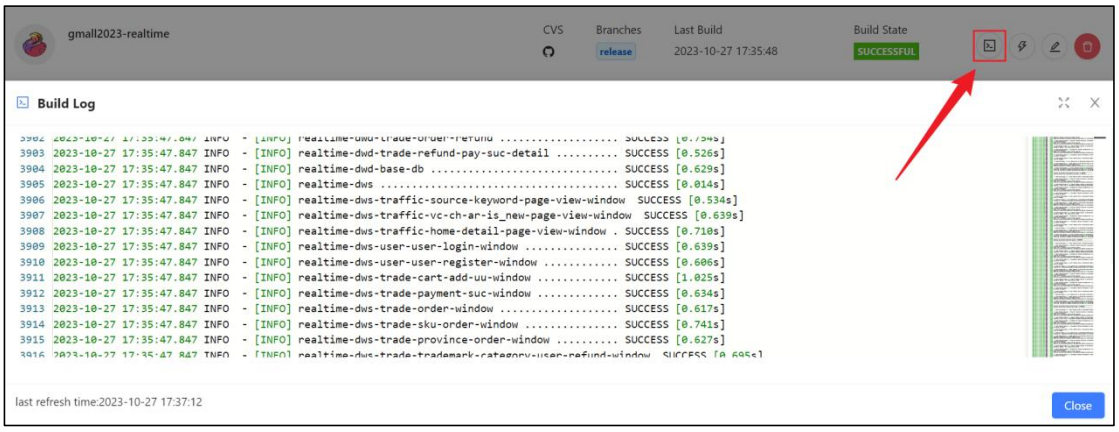


3）添加项目信息

3.2.3 编译项目

1）首次编译整个工程

2）查看日志



3）查看项目状态

编译成功。

3.2.4 拷贝 common 包

1）查看 StreamPark 目录

（1）本地编译目录

StreamPark 会把代码从 GitLab 拉取到服务器上，打好的包放在 ditst 目录中。

```
[atguigu@hadoop102 dist]$ ll /opt/module/streampark_workspace/dist/100001/
```

结果如下

```
[atguigu@hadoop102 dist]$ ll /opt/module/streampark_workspace/dist/100001/
总用量 0
drwxrwxr-x 2 atguigu atguigu 46 10月 27 17:42 realtime-common-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 43 10月 27 17:42 realtime-dim-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 51 10月 27 17:42 realtime-dwd-base-db-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 52 10月 27 17:42 realtime-dwd-base-log-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 68 10月 27 17:42 realtime-dwd-interaction-comment-info-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 58 10月 27 17:42 realtime-dwd-trade-cart-add-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 69 10月 27 17:42 realtime-dwd-trade-order-cancel-detail-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 62 10月 27 17:42 realtime-dwd-trade-order-detail-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 70 10月 27 17:42 realtime-dwd-trade-order-pay-suc-detail-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 62 10月 27 17:42 realtime-dwd-trade-order-refund-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 71 10月 27 17:42 realtime-dwd-trade-refund-pay-suc-detail-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 68 10月 27 17:42 realtime-dws-trade-cart-add-uu-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 62 10月 27 17:42 realtime-dws-trade-order-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 68 10月 27 17:42 realtime-dws-trade-payment-suc-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 71 10月 27 17:42 realtime-dws-trade-province-order-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 66 10月 27 17:42 realtime-dws-trade-sku-order-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 87 10月 27 17:42 realtime-dws-trade-trademark-category-user-refund-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 80 10月 27 17:42 realtime-dws-traffic-home-detail-page-view-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 83 10月 27 17:42 realtime-dws-traffic-source-keyword-page-view-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 84 10月 27 17:42 realtime-dws-traffic-vc-ch-ar-is_new-page-view-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 66 10月 27 17:42 realtime-dws-user-user-login-window-1.0-SNAPSHOT
drwxrwxr-x 2 atguigu atguigu 69 10月 27 17:42 realtime-dws-user-user-register-window-1.0-SNAPSHOT
```

（2）本地项目源码目录

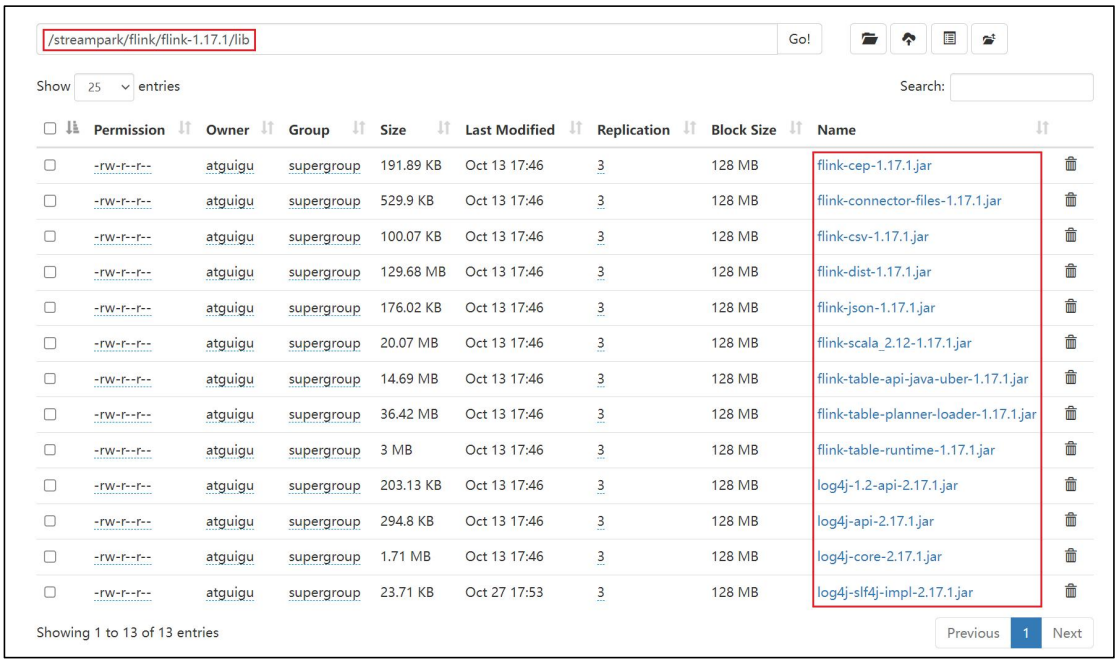
```
[atguigu@hadoop102 ~]$ ll /opt/module/streampark_workspace/project/
```

结果如下

```
[atguigu@hadoop102 ~]$ ll /opt/module/streampark_workspace/project/
总用量 0
drwxrwxr-x 3 atguigu atguigu 40 10月 27 17:35 gmall2023-realtime
drwxrwxr-x 2 atguigu atguigu  6 10月 13 16:01 streampark-quickstart
[atguigu@hadoop102 ~]$
```

（3）将本地的 Flink-1.17.1 目录上传到 HDFS 的/streampark/flink/目录

```
hadoop fs -mkdir -p /streampark/flink/
hadoop fs -put /opt/module/flink-1.17.1 /streampark/flink/
```



2）将编译好的 common 包拷贝至 Flink 的 lib 目录下

（1）拷贝至本地目录

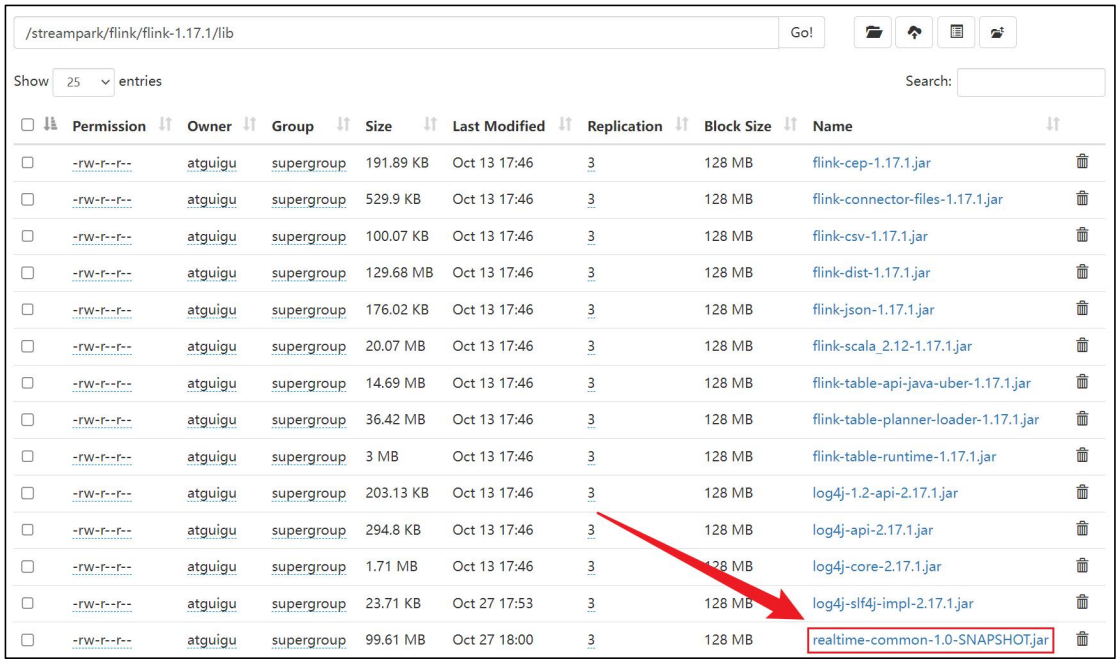
```
[atguigu@hadoop102 ~]$ cp
/opt/module/streampark_workspace/dist/100001/realtime-common-1.0-SNAPSHOT/re
```

```
altime-common-1.0-SNAPSHOT.jar /opt/module/flink-1.17.1/lib/
```

（2）上传至 HDFS

```
[atguigu@hadoop102 ~]$ hadoop fs -put
/opt/module/streampark_workspace/dist/100001/realtime-common-1.0-SNAPSHOT/re
altime-common-1.0-SNAPSHOT.jar /streampark/flink/flink-1.17.1/lib
```

查看 HDFS 目标路径



3.2.5 编译单个模块

如果修改了某个模块的代码，可以单独编译该模块，假设已完成代码修改并提交至 GitLab。

1）修改项目配置

将 POM 依赖配置为需要编译的子模块 POM 文件

此处的路径为以父工程为基准的相对路径。

2）重新编译

3）查看日志

```
Build Log

14 2023-10-27 18:07:29.596 INFO  -  / .git
15 2023-10-27 18:07:29.596 INFO  - [StreamPark] project [gmall2023-realtime] git clone successful!
16
17 2023-10-27 18:07:30.132 INFO  - [INFO] Scanning for projects...
18 2023-10-27 18:07:30.177 INFO  - [INFO]
19 2023-10-27 18:07:30.177 INFO  - [INFO] ------------------------------------------------------------------------
20 2023-10-27 18:07:30.177 INFO  - [INFO] Building realtime-dim 1.0-SNAPSHOT
21 2023-10-27 18:07:30.177 INFO  - [INFO] ------------------------------------------------------------------------
22 2023-10-27 18:07:30.787 INFO  - [INFO]
23 2023-10-27 18:07:30.787 INFO  - [INFO] --- maven-clean-plugin:2.4.1:clean (default-clean) @ realtime-dim ---

last refresh time:2023-10-27 18:07:36
```

可以看到，此处只编译了目标模块。

3.2.6 配置作业

1）注释代码中的环境配置

Flink 作业的并行度等配置信息可以代码中指定，也可以在提交时指定，前者的优先级更高，部署前我们应该注释 BaseApp 和 BaseSQLApp 中的相关配置，提交至 GitLab，重新编译，更新本地和 HDFS 的 common 包。这样，更改配置信息只需要修改提交参数，不必重新执行上述流程。

（1）追踪 release 分支

打开 git 命令行终端

① 获取远端分支信息

$ git fetch

```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realti
me2023/gmall2023-realtime (dev)
$ git fetch
From hadoop104:rdc/bigdata/gmall2023-realtime
 * [new branch]      release    -> gmall2023-realtime/release
```

② 查看远端分支

$ git branch --remote

```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realti
me2023/gmall2023-realtime (dev)
$ git branch --remote
  gmall2023-realtime/dev
  gmall2023-realtime/master
  gmall2023-realtime/release
```

③ 追踪远端分支

$ git checkout release

```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realti
me2023/gmall2023-realtime (dev)
$ git checkout release
Switched to a new branch 'release'
M       realtime-common/src/main/java/com/atguigu/gmall/realtime/common/base/Bas
eApp.java
M       realtime-common/src/main/java/com/atguigu/gmall/realtime/common/base/Bas
eSQLApp.java
branch 'release' set up to track 'gmall2023-realtime/release'.
```

（2）修改 BaseApp 的 start 方法

```
public void start(int port, int parallelism, String ckAndGroupId, String topic) {
    // 1. 环境准备
    // 1.1 设置操作 Hadoop 的用户名为 Hadoop 超级用户 atguigu
    System.setProperty("HADOOP_USER_NAME", "atguigu");

    // 1.2 获取流处理环境，并指定本地测试时启动 WebUI 所绑定的端口
    Configuration conf = new Configuration();
    conf.setInteger("rest.port", port);
    StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment(conf);

    // 1.3 设置并行度
    /*
    env.setParallelism(parallelism);

    // 1.4 状态后端及检查点相关配置
    // 1.4.1 设置状态后端
    env.setStateBackend(new HashMapStateBackend());

    // 1.4.2 开启 checkpoint
    env.enableCheckpointing(5000);
    // 1.4.3 设置 checkpoint 模式: 精准一次

env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.EXACTLY_ONCE);
    // 1.4.4 checkpoint 存储

env.getCheckpointConfig().setCheckpointStorage("hdfs://hadoop102:8020/gmall2023/stream/" +
ckAndGroupId);
    // 1.4.5 checkpoint 并发数
    env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);
    // 1.4.6 checkpoint 之间的最小间隔
    env.getCheckpointConfig().setMinPauseBetweenCheckpoints(500);
    // 1.4.7 checkpoint 的超时时间
    env.getCheckpointConfig().setCheckpointTimeout(10000);
    // 1.4.8 job 取消时 checkpoint 保留策略
```

```
env.getCheckpointConfig().setExternalizedCheckpointCleanup(RETAIN_ON_CANCELLATION)
;
        */

        // 1.5 从 Kafka 目标主题读取数据，封装为流
        KafkaSource<String> source = FlinkSourceUtil.getKafkaSource(ckAndGroupId, topic);

        DataStreamSource<String> stream = env.fromSource(source,
WatermarkStrategy.noWatermarks(), "kafka_source");

        // 2. 执行具体的处理逻辑
        handle(env, stream);

        // 3. 执行 Job
        try {
            env.execute();
        } catch (Exception e) {
            e.printStackTrace();
        }
}
```

（3）修改 BaseSQLApp 的 start 方法

```
public void start(int port,
                  int parallelism,
                  String ck) {
        System.setProperty("HADOOP_USER_NAME", "atguigu");

        Configuration conf = new Configuration();
        conf.setInteger("rest.port", port);
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment(conf);
        /*
        env.setParallelism(parallelism);
        // 1. 设置状态后端
        env.setStateBackend(new HashMapStateBackend());

        // 2. 开启 checkpoint
        env.enableCheckpointing(5000);
        // 3. 设置 checkpoint 模式: 精准一次

env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.EXACTLY_ONCE);
        // 4. checkpoint 存储
        env.getCheckpointConfig().setCheckpointStorage("hdfs://hadoop102:8020/gmall2023/sql/" +
ck);
```

```
    // 5. checkpoint 并发数
    env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);
    // 6. checkpoint 之间的最小间隔
    env.getCheckpointConfig().setMinPauseBetweenCheckpoints(500);
    // 7. checkpoint 的超时时间
    env.getCheckpointConfig().setCheckpointTimeout(10000);
    // 8. job 取消的时候的, checkpoint 保留策略

env.getCheckpointConfig().setExternalizedCheckpointCleanup(RETAIN_ON_CANCELLATION)
;
     */

    StreamTableEnvironment tEnv = StreamTableEnvironment.create(env);

    handle(env, tEnv);

}
```

（4）提交代码至本地仓库

$ git **add** *

```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realti
me2023/gmall2023-realtime (release)
$ git add *
The following paths are ignored by one of your .gitignore files:
target
hint: Use -f if you really want to add them.
hint: Turn this message off by running
hint: "git config advice.addIgnoredFile false"
```

$ git commit -m "部署前注释代码中的配置操作"

```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realtime2023/gmall2023-real
time (release)
$ git commit -m "部署前注释代码中的配置操作"
[release c2fe7b7] 部署前注释代码中的配置操作
 2 files changed, 4 insertions(+)
```

（5）提交代码至远端

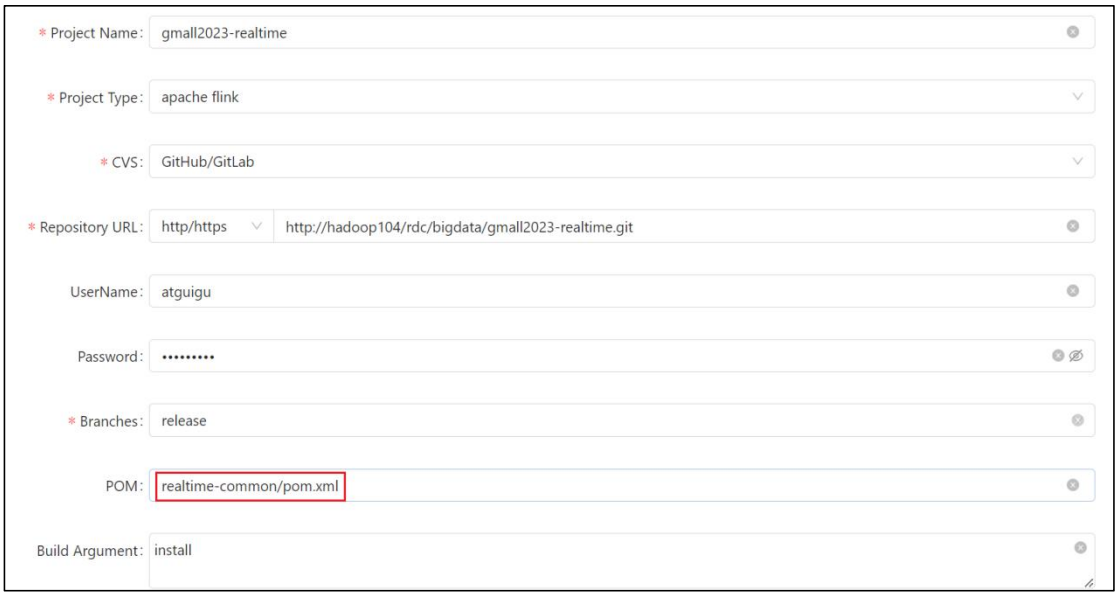
$ git **push**

```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realtime2023/gmall2023-real
time (release)
$ git push
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 20 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (14/14), 1.05 KiB | 1.05 MiB/s, done.
Total 14 (delta 4), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for release, visit:
remote:   http://hadoop104/rdc/bigdata/gmall2023-realtime/-/merge_requests/new?merge_request%5Bsource
_branch%5D=release
remote:
To hadoop104:rdc/bigdata/gmall2023-realtime.git
   a8bd469..c2fe7b7  release -> release
```

2）重新编译 common 模块

修改了 common 模块代码，需要重新编译生效

（1）修改 POM 路径



（2）重新编译

（3）完成

```
14 2023-10-30 11:12:16.550 INFO  -  / .git
15 2023-10-30 11:12:16.550 INFO  - [StreamPark] project [gmall2023-realtime] git clone successful!
16
17 2023-10-30 11:12:17.044 INFO  - [INFO] Scanning for projects...
18 2023-10-30 11:12:17.082 INFO  - [INFO]
19 2023-10-30 11:12:17.082 INFO  - [INFO] ------------------------------------------------------------------------
20 2023-10-30 11:12:17.082 INFO  - [INFO] Building realtime-common 1.0-SNAPSHOT
21 2023-10-30 11:12:17.082 INFO  - [INFO] ------------------------------------------------------------------------
22 2023-10-30 11:12:18.431 INFO  - [INFO]
23 2023-10-30 11:12:18.431 INFO  - [INFO] --- maven-clean-plugin:2.4.1:clean (default-clean) @ realtime-common ---
24 2023-10-30 11:12:18.459 INFO  - [INFO]
25 2023-10-30 11:12:18.459 INFO  - [INFO] --- maven-resources-plugin:2.5:resources (default-resources) @ realtime-common ---
26 2023-10-30 11:12:18.514 INFO  - [debug] execute contextualize
```

```
2532 2023-10-30 11:12:41.306 INFO  - [INFO] --- maven-install-plugin:2.3.1:install (default-install) @ realtime-common ---
2533 2023-10-30 11:12:41.326 INFO  - [INFO] Installing /opt/module/streampark_workspace/project/gmall2023-realtime/gmall2023-realtime-release/realtime-common/ta
2534 2023-10-30 11:12:41.532 INFO  - [INFO] Installing /opt/module/streampark_workspace/project/gmall2023-realtime/gmall2023-realtime-release/realtime-common/de
2535 2023-10-30 11:12:41.540 INFO  - [INFO] ------------------------------------------------------------------------
2536 2023-10-30 11:12:41.540 INFO  - [INFO] BUILD SUCCESS
2537 2023-10-30 11:12:41.540 INFO  - [INFO] ------------------------------------------------------------------------
2538 2023-10-30 11:12:41.540 INFO  - [INFO] Total time: 24.504s
2539 2023-10-30 11:12:41.540 INFO  - [INFO] Finished at: Mon Oct 30 11:12:41 CST 2023
2540 2023-10-30 11:12:41.673 INFO  - [INFO] Final Memory: 200M/1539M
2541 2023-10-30 11:12:41.673 INFO  - [INFO] ------------------------------------------------------------------------
2542 2023-10-30 11:12:41.710 INFO  - update deploy by project: gmall2023-realtime, appName:DIM层维度分流
2543
```

3）重新上传 realtime-common-1.0-SNAPSHOT.jar

（1）替换 HDFS /streampark/flink/flink-1.17.1/lib 的 realtime-common-1.0-SNAPSHOT.jar

略。

（2）替换本地/opt/module/flink-1.17.1/lib/下的 realtime-common-1.0-SNAPSHOT.jar

更多 Java –大数据 –前端 –python 人工智能资料下载，可百度访问：尚硅谷官网

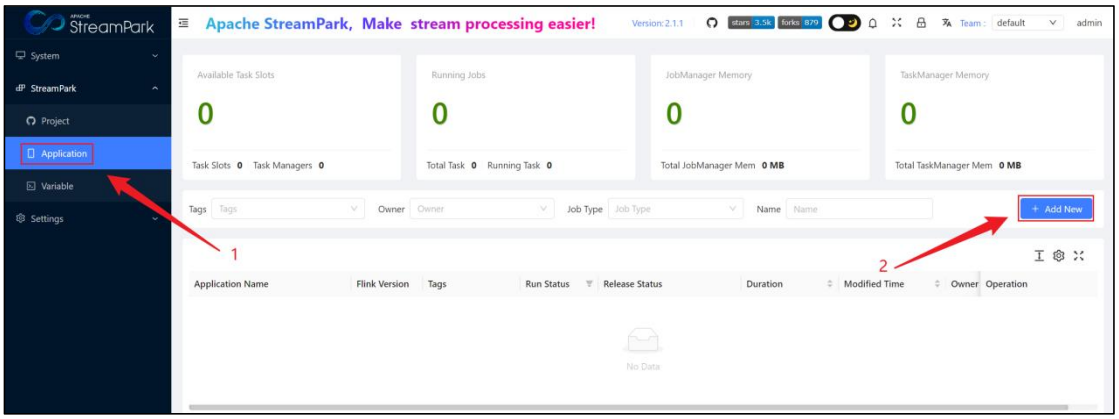
略。

4）修改 POM 路径

（1）将 POM 路径修改为业务 module POM 路径。

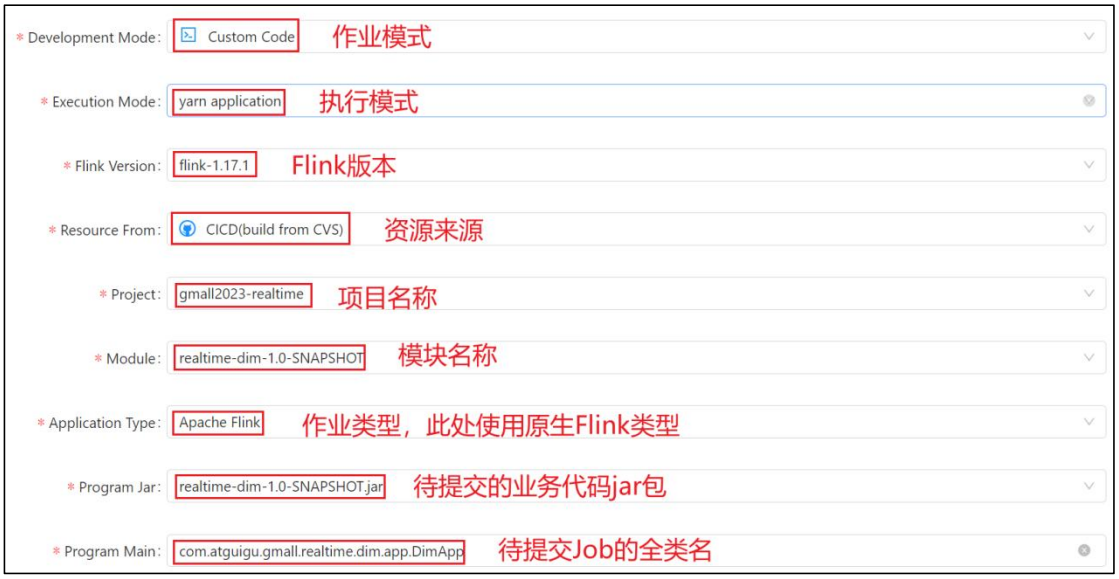
（2）重新编译

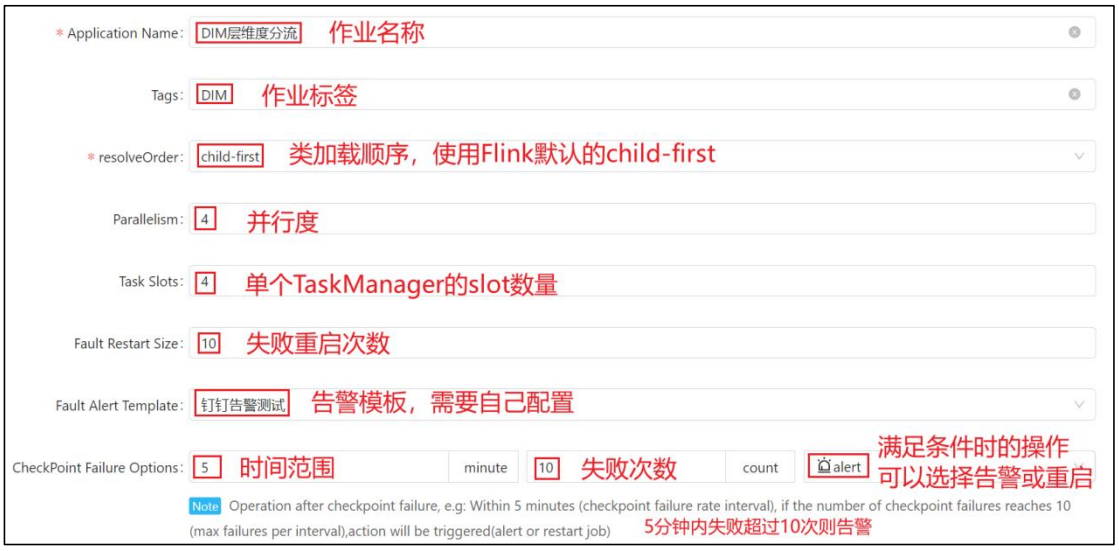
5）新建作业



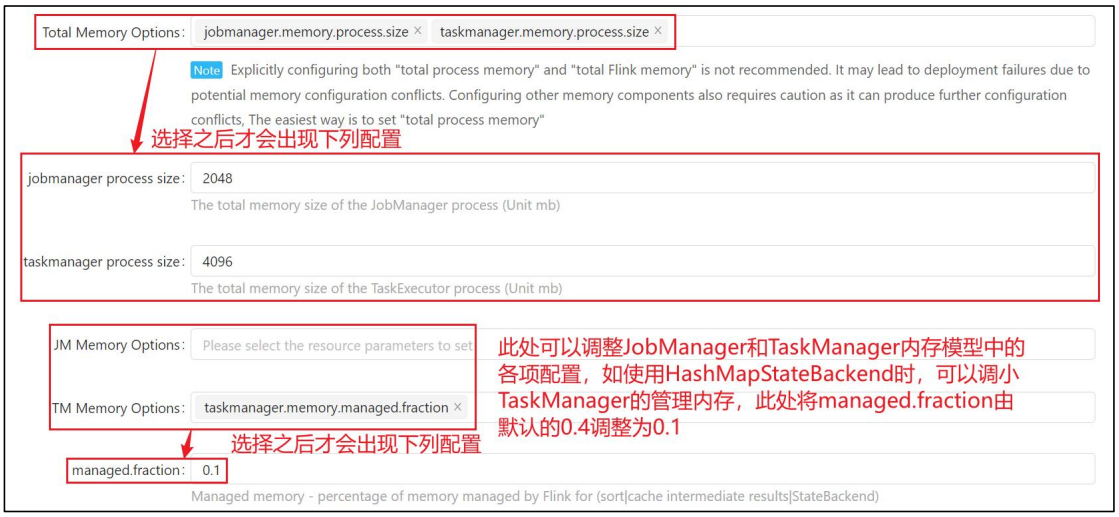
6）配置作业

（1）基本配置




* Application Name: DIM层维度分流 作业名称
Tags: DIM 作业标签
* resolveOrder: child-first 类加载顺序，使用Flink默认的child-first
Parallelism: 4 并行度
Task Slots: 4 单个TaskManager的slot数量
Fault Restart Size: 10 失败重启次数
Fault Alert Template: 钉钉告警测试 告警模板，需要自己配置
CheckPoint Failure Options: 5 时间范围 minute 10 失败次数 count alert
满足条件时的操作
可以选择告警或重启
Note Operation after checkpoint failure, e.g: Within 5 minutes (checkpoint failure rate interval), if the number of checkpoint failures reaches 10 (max failures per interval),action will be triggered(alert or restart job)
5分钟内失败超过10次则告警

（2）内存配置

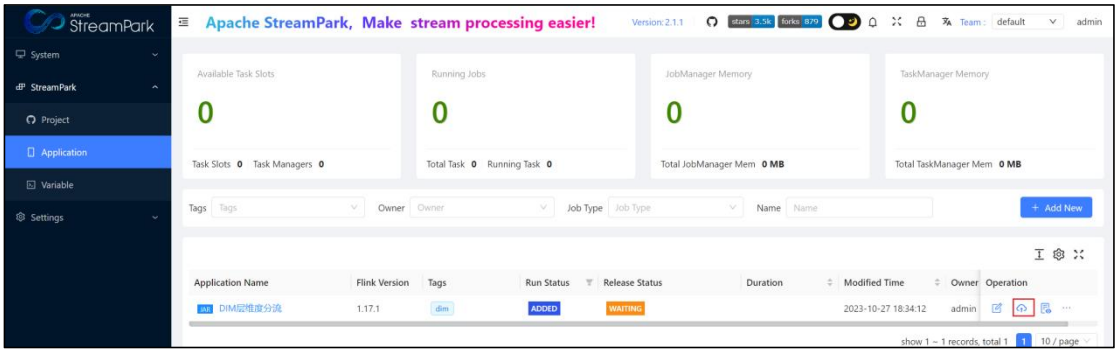

Total Memory Options: jobmanager.memory.process.size × taskmanager.memory.process.size ×
Note Explicitly configuring both "total process memory" and "total Flink memory" is not recommended. It may lead to deployment failures due to potential memory configuration conflicts. Configuring other memory components also requires caution as it can produce further configuration conflicts, The easiest way is to set "total process memory"
选择之后才会出现下列配置
jobmanager process size: 2048
The total memory size of the JobManager process (Unit mb)
taskmanager process size: 4096
The total memory size of the TaskExecutor process (Unit mb)
JM Memory Options: Please select the resource parameters to set
TM Memory Options: taskmanager.memory.managed.fraction ×
此处可以调整JobManager和TaskManager内存模型中的各项配置，如使用HashMapStateBackend时，可以调小TaskManager的管理内存，此处将managed.fraction由默认的0.4调整为0.1
选择之后才会出现下列配置
managed.fraction: 0.1
Managed memory - percentage of memory managed by Flink for (sort|cache intermediate results|StateBackend)

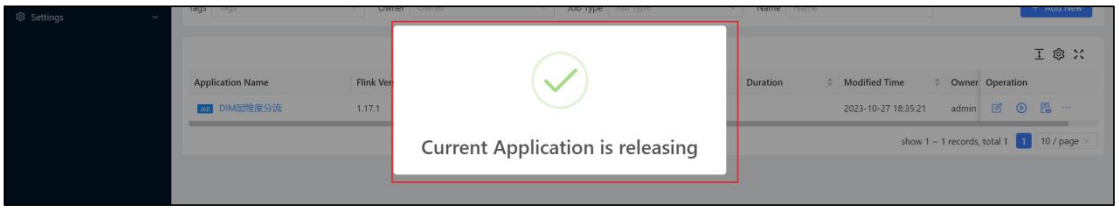
（3）其它配置

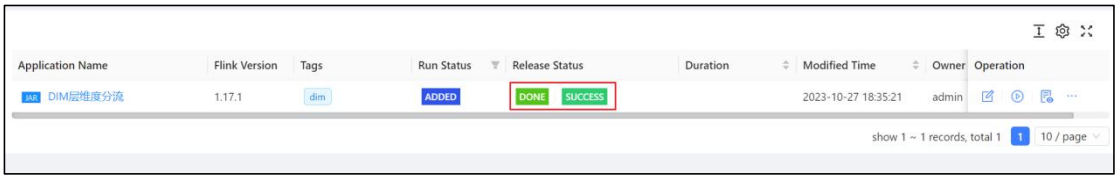
Yarn队列: default
指定配置过的yarn队列
提示 用来快速设置 'yarn.application.name' 与 'yarn.application.node-label'。如果没有可用队列请联系管理员在 'Settings' -> 'Yarn Queue' 页面中创建

3.2.7 发布和启动作业

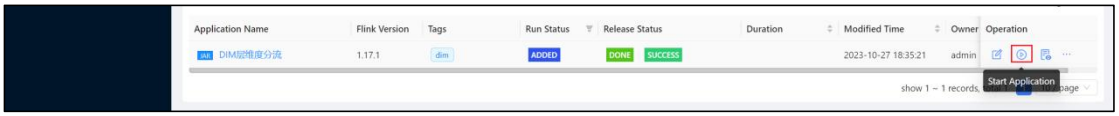
1）发布作业



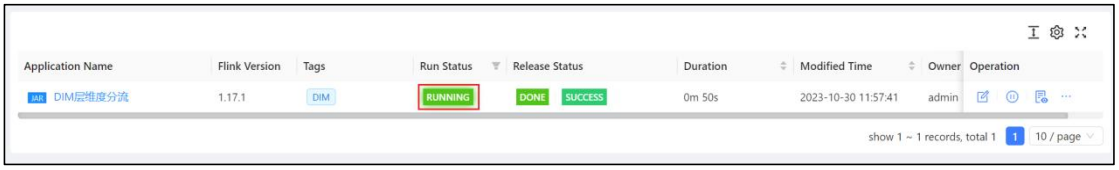




2）启动作业



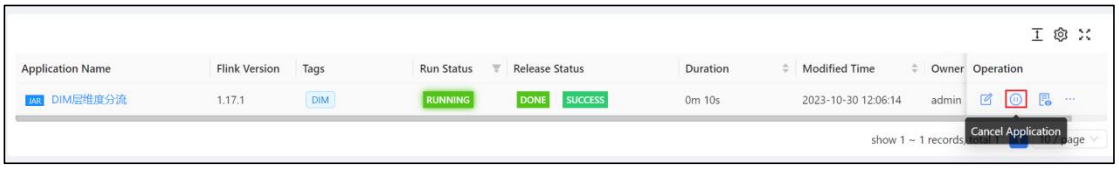
3）查看作业状态

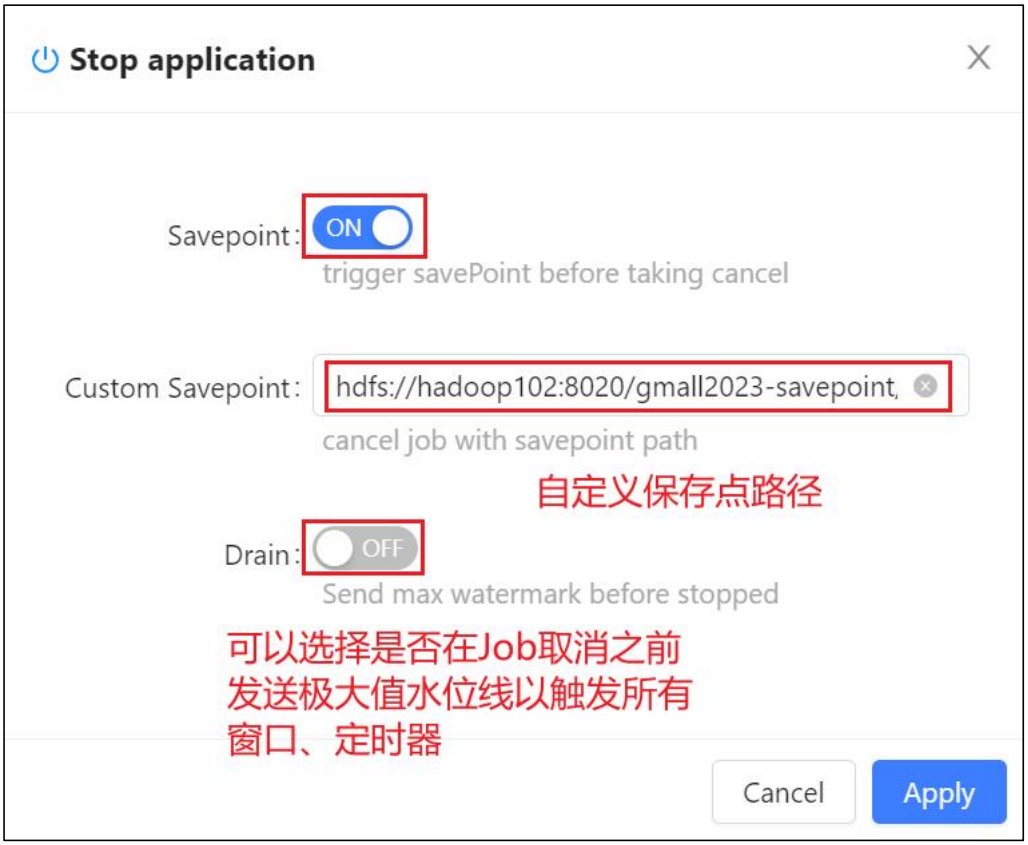


作业正在运行，部署完成。

4）取消 Job 并触发保存点

（1）取消 Job

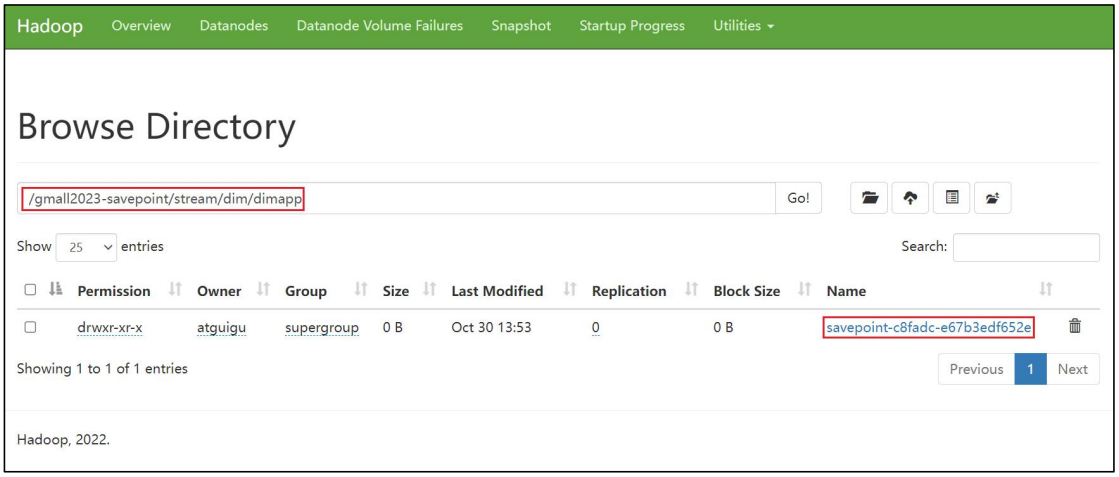




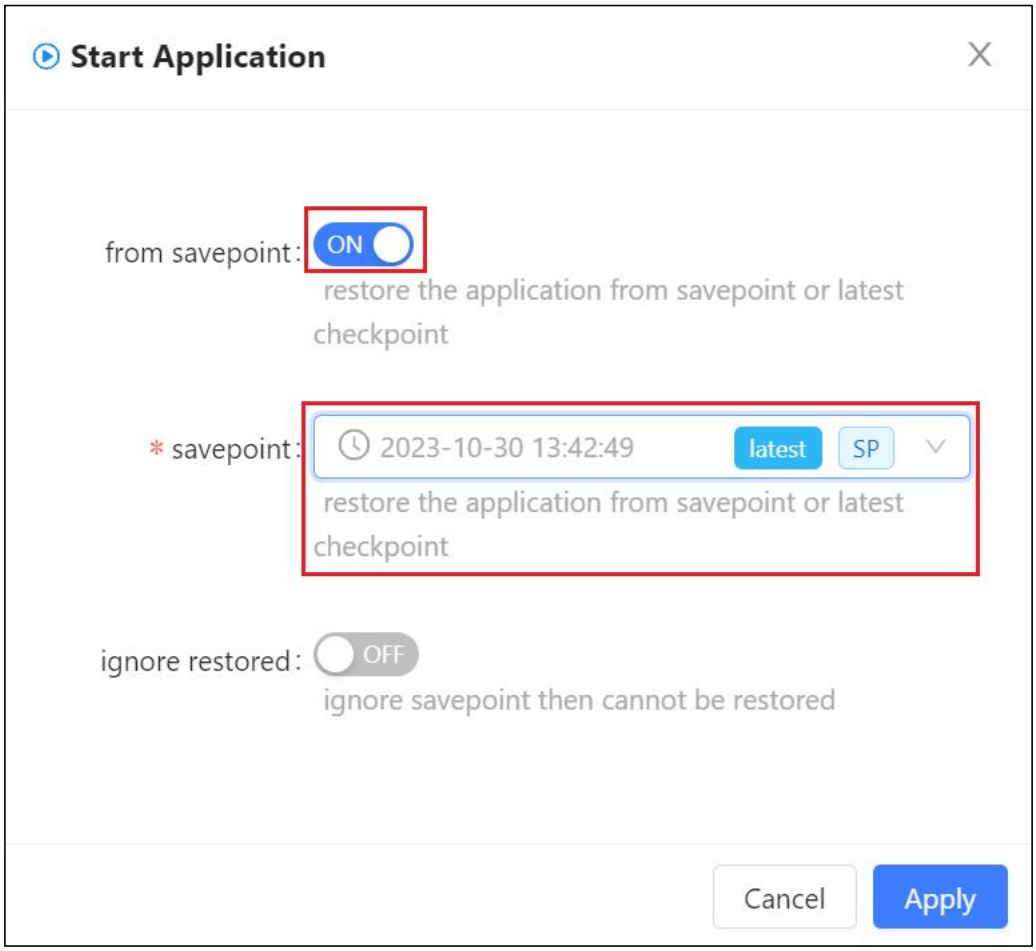
（2）正在取消 Job 并触发保存点

（3）Job 已取消

（4）查看保存点路径

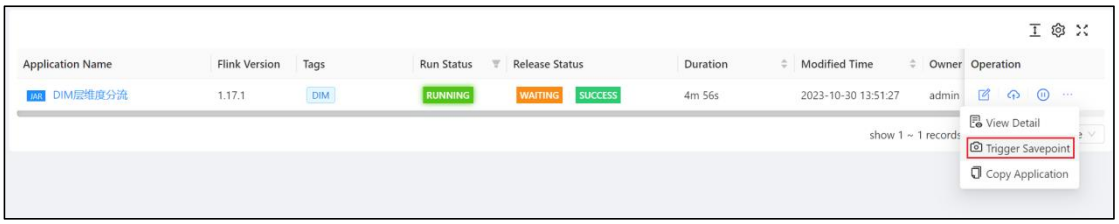


（5）触发保存点后，再次启动，可以从保存点启动

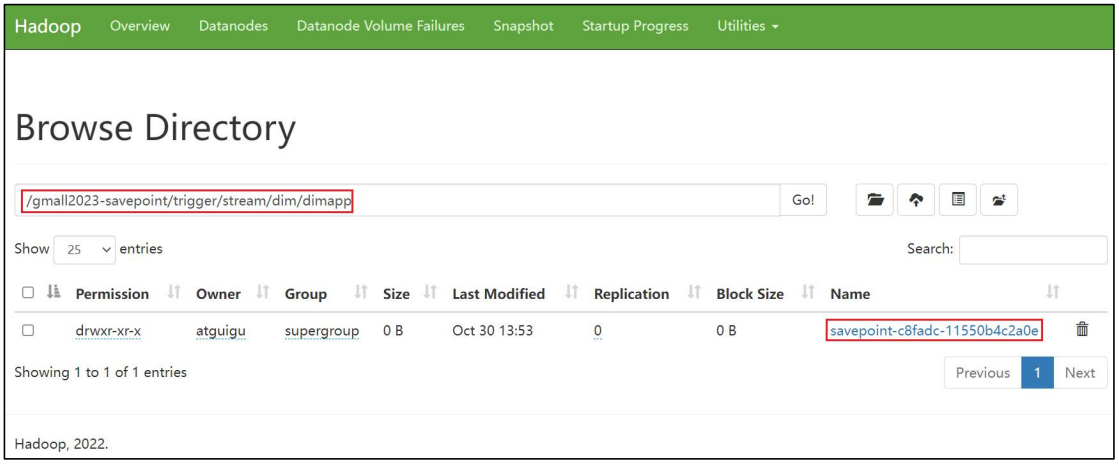


5）触发保存点但不停止 Job

（1）触发保存点



（2）查看目标路径



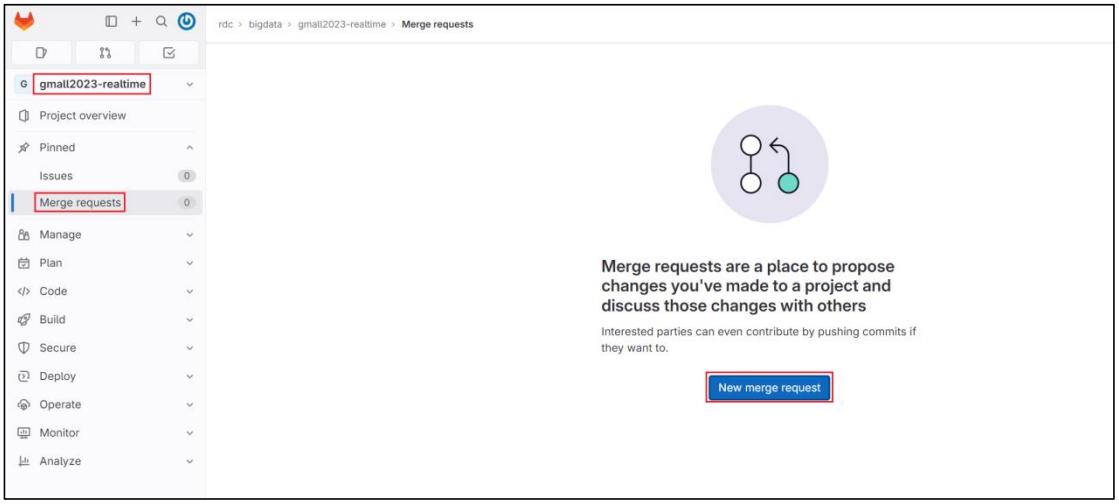
3.2.8 合并至 master 和 dev 分支

按照上述流程依次部署测试每个 Job，此处不再一一演示。

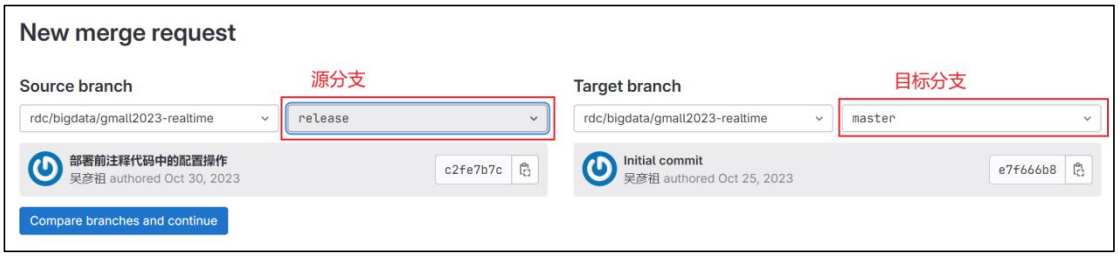
假设我们已经完成了所有 Job 的测试，接下来准备正式发布 1.0 版本。

1）将 release 分支合并至 master 分支

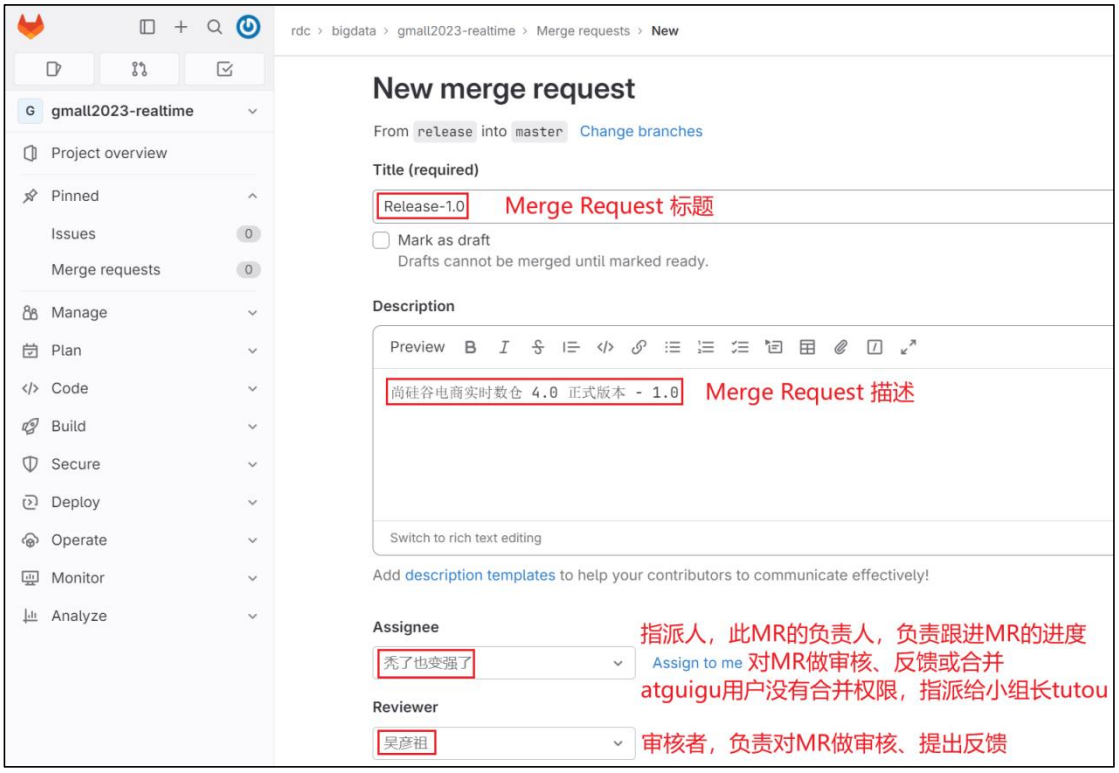
（1）新建 Merge Request



（2）选择源分支和目标分支

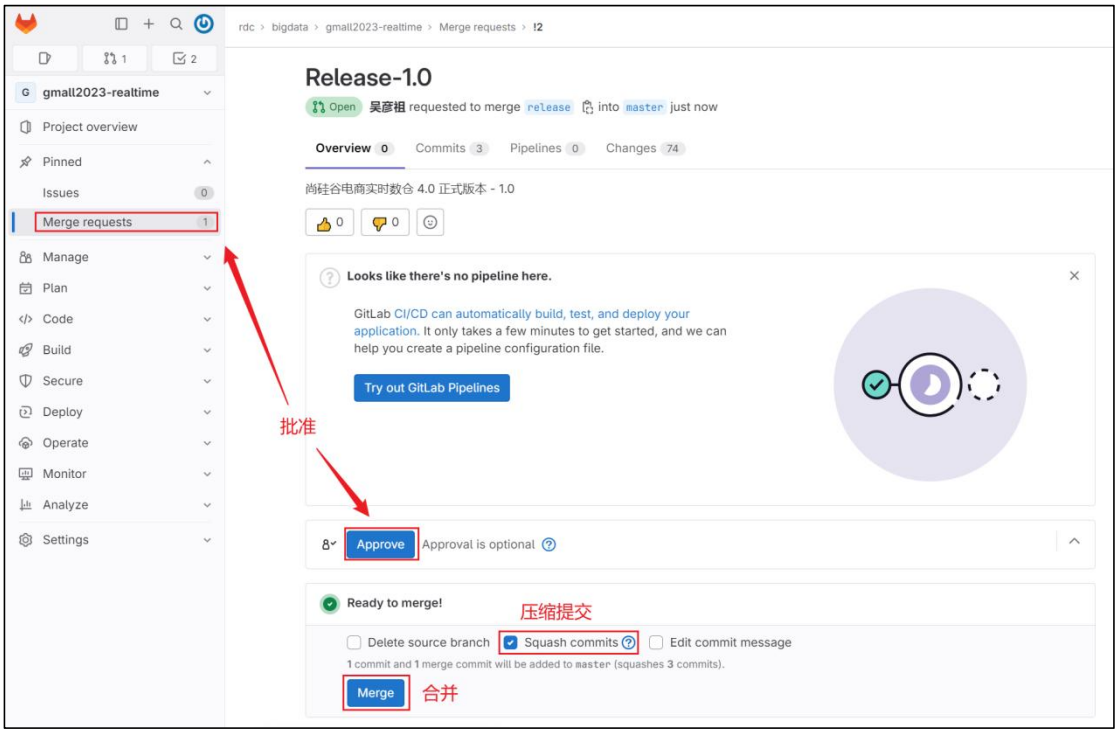


（3）配置 Merge Request

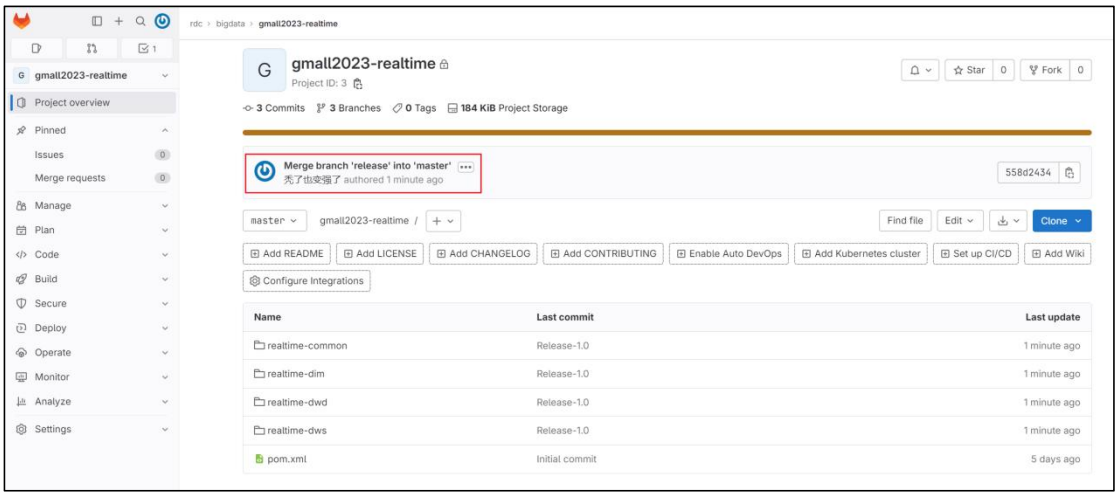


（4）批准 Merge Request，完成合并

切换到 tutou 用户，批准 Merge Request。



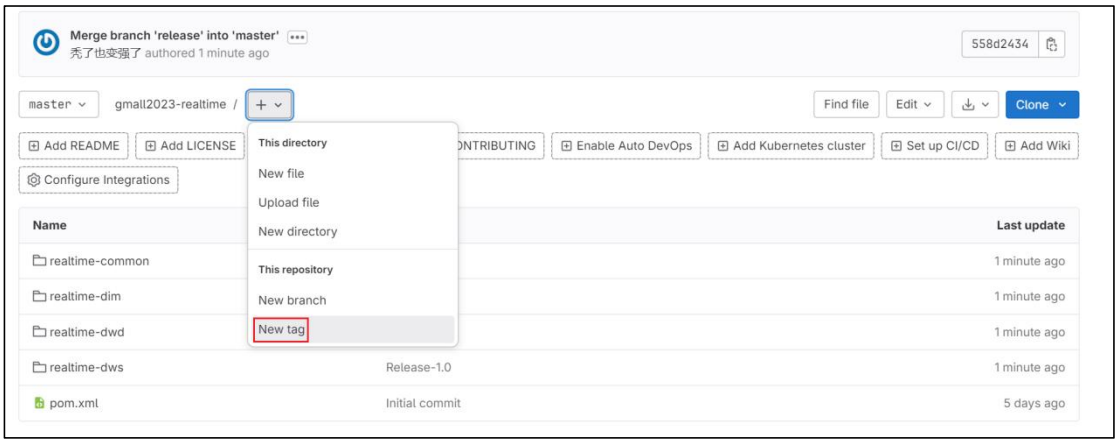
（5）查看 master 分支



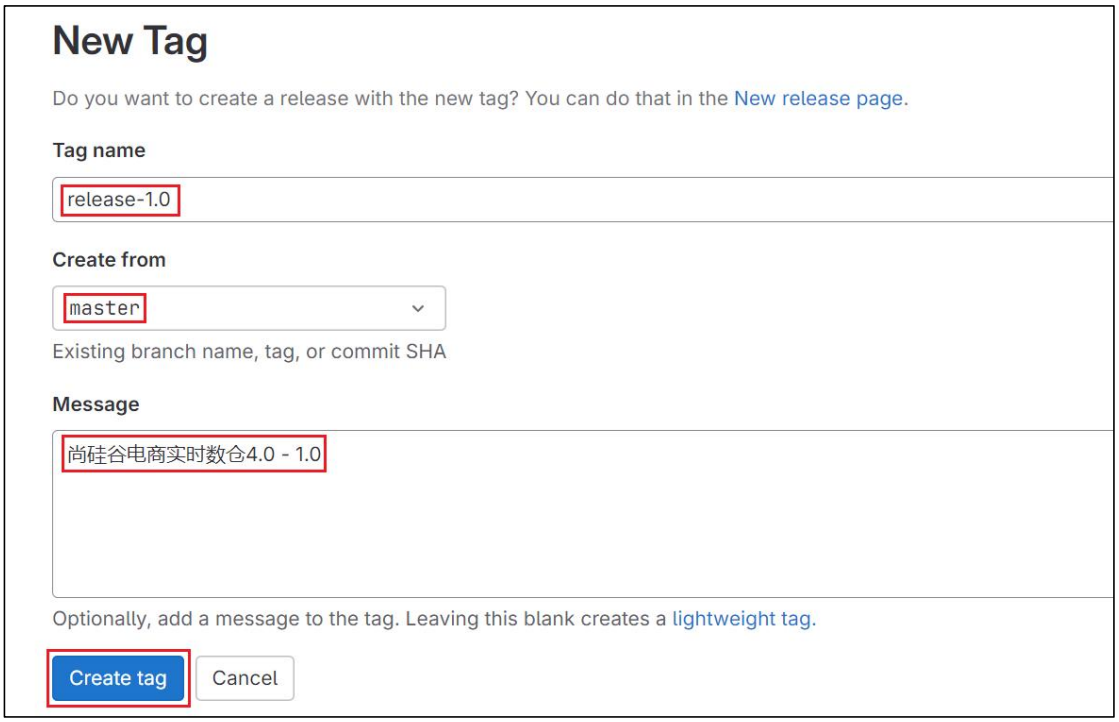
已将 release 分支合并至 master 分支

（6）打标签

① 新建标签



② 配置标签

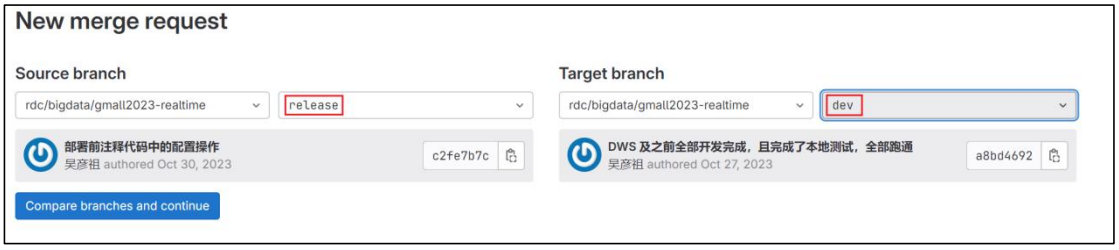


③ 查看已完成的标签

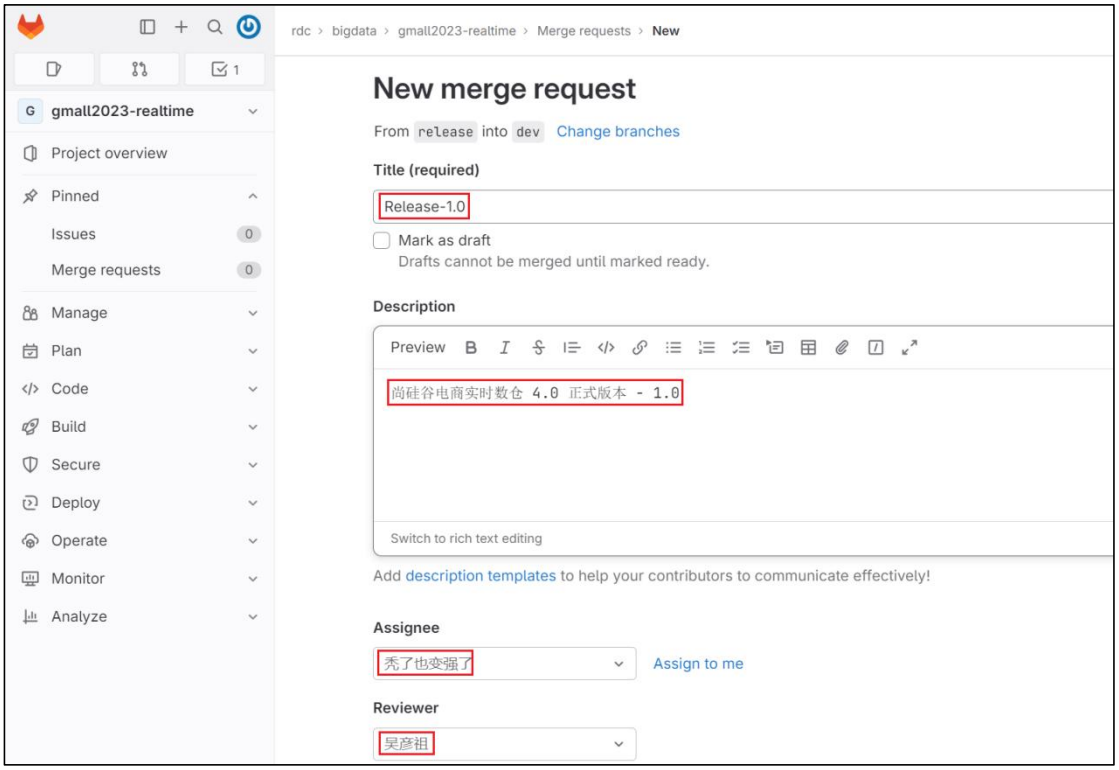
2）将 release 分支合并至 dev 分支

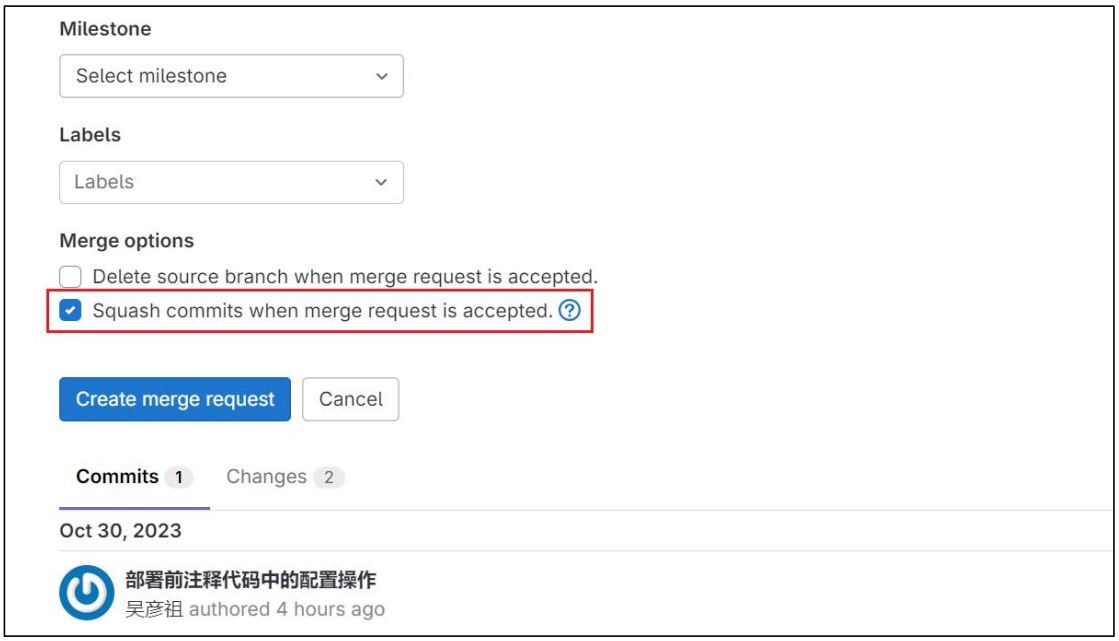
回到 atguigu 用户

（1）新建 MR



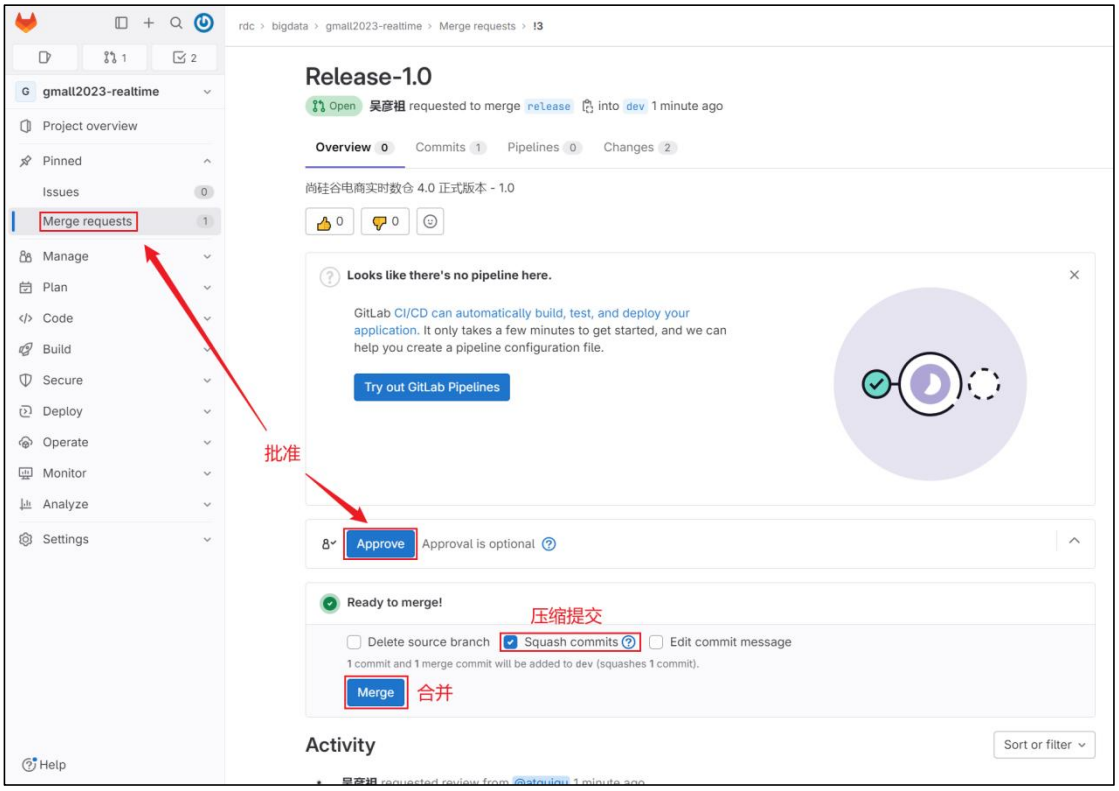
（2）配置 MR



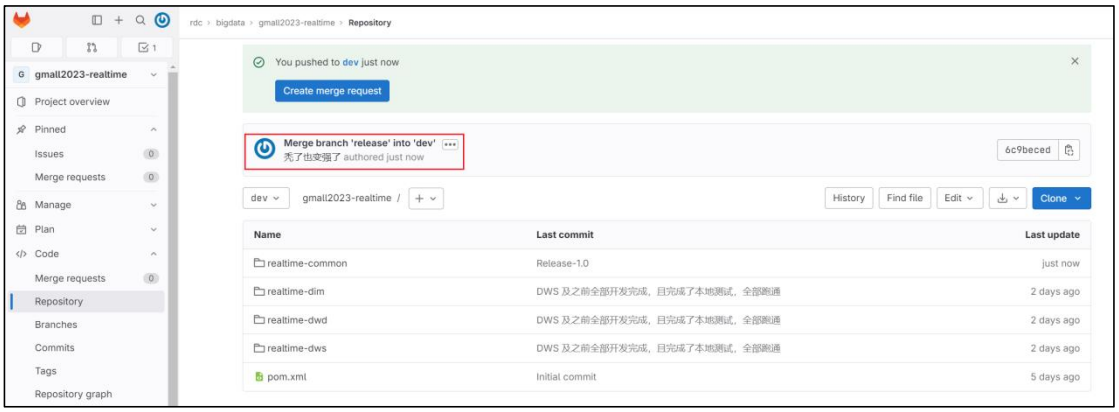


（3）批准 MR，完成合并

切换到 tutou 用户



（4）查看 dev 分支



已完成合并